

PA3 实验报告

231180166 杜明阳

实验进度

除了极少数可选题，PA3实验任务全部完成，实现了所有要求的系统调用，构建了简易文件系统，能够流畅运行 pal，批处理系统也很完善了

必答题

重新组织Context结构体

__am_irq_handle() 的 Context *c 是怎么来的？每个成员在哪里赋值？各部分关系？

结构体来源和成员赋值流程：

- riscv.h

定义了 struct Context，成员包括通用寄存器数组 gpr[]、pdir、mcause、mstatus、mepc。

- trap.S

异常发生时，trap.S 汇编代码会：依次用 sw 指令把所有通用寄存器压栈（赋值到 gpr[]）。把 mcause、mstatus、mepc 也压栈（赋值到对应成员）。地址空间信息 pdir 通常和 gpr[0] 共用空间（x0不用保存实际值）。最后把栈指针作为 Context *c 传给 __am_irq_handle(c)。

- inst.c 的指令

指令如 ecall 触发异常，导致进入 trap.S。CSR 指令（如 csrw, csrrw）会影响 mepc、mcause、mstatus 等 CSR 的值，这些值会被 trap.S 保存到 Context。

- 总结

Context 的每个成员都是在 trap.S 里通过保存寄存器和 CSR 寄存器赋值的。__am_irq_handle(Context *c) 里拿到的 c，就是 trap.S 构造好的上下文快照。

关系梳理

riscv.h：定义了上下文结构体和成员顺序。trap.S：异常入口，负责保存所有寄存器和CSR，构造 Context，并调用 C 层处理函数。inst.c：负责指令解释，触发异常（如 ecall），影响 CSR 的值。__am_irq_handle：C层异常分发处理，拿到 trap.S 构造的 Context，可以访问所有寄存器和异常信息。

从yield test调用yield()开始, 到从yield()返回的期间, 这一趟旅程具体经历了什么?

概览

应用 (yield test) 执行 yield() → 生成 ecall 指令 → 指令解码 (inst.c) 触发 isa_raise_intr(...) → 模拟器 (NEMU) 设置 CSR / 改变 PC 到 mtvec → 在 mtvec 地址执行 trap.S (汇编) 保存寄存器并构造 Context → 调用 C 层 __am_irq_handle(Context *c) → 事件识别并调用 user_handler(simple_trap) → 调整 c->mepc (+4) 以便返回下一条指令 → __am_irq_handle 返回 → trap.S 恢复寄存器并执行 mret → 在应用层继续执行 yield() 之后的指令。

下面逐步细化每一步的细节。

阶段 0 — 前提与坐标

寄存器编号与约定：按 RISC-V, a7 = x17 (非 E 架构), a5 = x15 (E 架构)。XLEN = 32 (4 字节)。Context 结构 (必须与 trap.S 保存顺序严格匹配)：

```
gpr[0..NR_REGS-1] (gpr[0] 可复用为 pdir), 紧接着
mcause
mstatus
mepc
```

关键 CSR (在 nemu/include/csr.h)：

```
CSR_MEPC = 0x341 (mepc)
CSR_MCAUSE = 0x342 (mcause)
CSR_MSTATUS = 0x300 (mstatus)
CSR_MTVEC = 0x305 (mtvec)
```

mtvec 被 cte_init() 通过 csrw mtvec, __am_asm_trap 设置为 trap.S 的入口地址。

阶段 1 — 应用层：yield() 生成异常

yield() 的实现 (C/ASM)：非 E：asm volatile("li a7, -1; ecall"); E：asm volatile("li a5, -1; ecall"); 这两条汇编的效果：li a7, -1 把特殊值 -1 放入寄存器 (用来在 __am_irq_handle 里区分 yield) ecall 发出环境调用同步异常 (trap) 指令编码：ecall 的机器码通常 0x00000073 (RISC-V base)。

阶段 2 — 指令解码/模拟：inst.c 与 isa_raise_intr

NEMU 的主循环 fetch -> decode -> execute。在 inst.c 的解码表中，ecall 的 INSTPAT 匹配到后执行的动作类似：s->pc 是当前 PC (指向 ecall 指令)。s->dnpc 是 next-pc (模拟器把它用作下一步要跳转的地址)。

isa_raise_intr(cause, pc) (模拟器/ISA 层实现) 做了这些事 (逻辑上)：写 CSR_MEPC = pc；(保存出异常时的 PC) 写 CSR_MCAUSE = cause；(例如 11) 修改 CSR_MSTATUS (保存/修改中断使能、MPP 等) — 以便后续 mret 恢复 返回 CSR_MTVEC 的值 (即 mtvec)，供上层把 PC 改为异常入口 NEMU 在执行 ecall 时会把 CPU 的 PC 设置为 isa_raise_intr 返回的 mtvec (模拟跳到 trap.S)，并继续在该地址执行指令 (即 trap.S 的汇编)。

阶段 3 — 进入 trap.S：保存上下文并构造 Context

trap.S 的关键作用：在软件层面模拟异常入口 prologue，把全部需要恢复的状态保存到栈 (或某个内存区)，并把栈指针作为 Context* 传给 __am_irq_handle。trap.S 的典型步骤 (以你工程的 trap.S 宏为准)：adjust sp：为保存空间分配栈帧 (通常 sub sp, sp, CONTEXT_SIZE) 保存通用寄存器 (按 REGLIST 宏顺序)：STORE x1, OFFSET(sp, r) 等 (用 sw 存每个寄存器) 注意：REGLIST 设计保证 gpr[0..NR_REGS-1] 在 Context 中有对应槽，x0 未保存实际值但其槽被保留 / 可复用为 pdir 保存 CSR 值到 context：从 CSR 读出 mcause, mstatus, mepc (或直接把已由 isa_raise_intr 写入的 CSR 值装载并存入栈对应位置) STORE mcause 到 OFFSET_CAUSE(sp) STORE mstatus 到 OFFSET_STATUS(sp) STORE mepc 到 OFFSET_EPC(sp) 因为 trap.S 的偏移定义通常为：OFFSET_CAUSE = NR_REGS*XLEN, 所以 mcause 紧跟在 gpr[] 之后 构造传参并调用 C 处理例程：把 sp (指向刚保

存的 Context) 放到 a0 (RISC-V 第一个整数参数寄存器) jal ra, __am_irq_handle (调用 C 函数) 结果：在进入 __am_irq_handle 时，C 层拿到的 Context *c 指向 trap.S 在栈上形成的上下文快照；c->gpr[]、c->mcause、c->mstatus、c->mepc 都已由 trap.S 写好。

阶段 4 — C 层处理：__am_irq_handle(Context *c)

文件：abstract-machine/.../cte.c 主要职责：根据 c->mcause 识别事件类型 (ecall、timer interrupt、device interrupt 等) 常见 mapping：mcause == 11 (ecall from M) or 8 (ecall from U) → ecall/软陷 (再看某寄存器是否为 -1 来区分 yield/syscall) mcause has high bit set (0x80000007 etc) → 外部中断/定时器 (按 RISC-V 约定) 为 ecall 类型进一步查看指定寄存器 (a7/x17 或 a5/x15) 是否等于 -1：if (c->gpr[17] == (uintptr_t)-1) ev.event = EVENT_YIELD; else ev.event = EVENT_SYSCALL; 对于同步 ecall/自陷，必须调整 c->mepc += 4 (或根据指令长度)：原因：RISC-V spec：mepc 保存的是触发异常的指令的地址。若要返回并执行异常后的“下一条指令”，需要把 mepc 加上指令长度 (riscv32 通常 +4)。在实现里：当 ev.event == EVENT_YIELD 或 EVENT_SYSCALL 时执行 c->mepc += 4。注意：这会修改栈上保存的 mepc 字段 (trap.S 在后续恢复/写回 CSR 时会使用这个新值)。调用用户注册的 handler (user_handler)，例如 simple_trap(Event ev, Context ctx)：simple_trap 依据 ev.event 输出 'y' 或 't' 或 'd'。user_handler 返回一个 Context (通常返回同一指针，可能修改部分寄存器以完成上下文切换)。__am_irq_handle 返回时，C 调用已完成并返回控制到 trap.S 的代码 (call 返回后在 trap.S 继续执行 epilogue)。

阶段 5 — trap.S 恢复与 mret

在 trap.S 的 call __am_irq_handle 返回处，接下来做：

可能将修改后的 mepc/mstatus 写回 CSR (具体实现可在 trap.S 或在 C 层前后实现)，但通常 trap.S 在保存时也把 CSR 保存到 context，恢复时用 context 的值写回：

STORE/LOAD 逆序：从 context 中加载寄存器值写回到 CPU 寄存器 (LOAD lw ...)

恢复栈 (add sp, sp, CONTEXT_SIZE)

执行 mret 指令 (汇编)：

mret 的语义 (硬件/模拟器实现)：从 CSR_MEPC 取值并把 PC 设置为该值；从 CSR_MSTATUS 恢复中断/特权位等。

在模拟器 (inst.c) 中 mret 可能由 INSPAT("0011000 00010 00000 000 00000 11100 11", mret, N, { s->dnpc = csr_read(CSR_MEPC); }) 处理：将 s->dnpc 设为 csr_mepc，从而下一 cycle 的 PC 就是保存的 mepc (我们在 __am_irq_handle 里已把 mepc += 4)。

结果：控制权回到应用层，PC = original_ecall_address + 4，应用继续从 yield() 的下一条指令执行 (或 yield() 返回，取决于实现)。

阶段 6 — 返回到应用、user 层结果可见

simple_trap 在 __am_irq_handle 被调用时输出 'y' (使用 putchar)，因此你会看到字符 y。

如果 simple_trap 只是打印字符而没有做复杂的上下文切换，应用继续运行原来的循环。

hello程序是什么, 它从而何来, 要到哪里去

1. hello.c 是怎么变成内存里的程序的？

- 编译与链接：hello.c 被编译为目标文件，再与 C 库、启动代码等链接成一个 ELF 格式的可执行文件。ELF 文件里包含了代码段、数据段、符号表等信息。

- 加载到内存：在 PA 的系统中，Nanos-lite（或更底层的 AM/NEMU）负责把 ELF 文件加载到物理内存的某个位置。这个过程通常由内核的 `naive_oload` 或类似函数完成：它解析 ELF 文件头，找到各个段（如 `.text`、`.data`），把它们复制到指定的内存地址。
 - 为什么是这个位置？：加载地址由链接脚本和系统约定决定，比如用户程序总是加载到 `0x83000000` 这样的高地址空间，避免和内核冲突。
2. 第一条指令在哪里？怎么执行到它？
- 入口点：ELF 文件头有一个 `entry` 字段，指明程序的入口地址（通常是 `_start` 或 `main` 的包装）。
 - CPU启动流程：系统启动后，内核初始化好环境（如堆栈、堆区），然后设置 PC（程序计数器）为 ELF 的入口地址，开始执行用户程序的第一条指令。
 - 上下文切换：在 AM/NEMU 平台，内核通过 `trap/interrupt` 机制切换到用户态，执行用户程序。
3. hello 程序如何不断打印字符串？
- 用户代码：hello.c 里调用 `write(1, "Hello World!\n", 13)` 和 `printf(...)`。
 - 系统调用：这些库函数最终会通过软中断（`ecall/syscall` 指令）进入内核，传递系统调用号（如 `SYS_write`）和参数（`fd`、`buf`、`len`）。
 - 内核处理：Nanos-lite 的 `do_syscall()` 识别出 `SYS_write`，判断 `fd=1`（`stdout`），然后循环调用 `putch()` 把每个字符输出到串口。
 - 串口输出：在 AM/NEMU 平台，`putch()` 最终调用底层的串口驱动（如 `outb(SERIAL_PORT, ch)`），把字符发送到终端（或仿真器的输出窗口）。
 - 终端显示：终端收到字符后显示出来，用户就看到了 "Hello World!"。
4. 每个字符的旅程
- 用户态：hello.c 里的 `write/printf` 触发系统调用。
 - 陷入内核：CPU 执行 `ecall`，进入 `trap handler`，内核解析 `syscall` 号和参数。
 - 内核分发：`do_syscall()` 处理 `SYS_write`，调用 `putch()`。
 - 硬件/仿真器：`putch()` 通过串口驱动把字符送到终端。
 - 用户看到：终端显示字符。

PAL动画背后

仙剑奇侠传启动动画（`PAL_SplashScreen`）播放仙鹤时，涉及了应用层库函数、`libos`、Nanos-lite、AM、NEMU 的协作。整个过程如下：

1. 应用层（`pal/main.c`）`PAL_SplashScreen()` 里调用了文件读写和图像绘制相关的库函数，如 `fopen`, `fread`, `fseek`, `PAL_DrawImage`, `PAL_UpdateScreen`。这些库函数会去读取 `mgo.mkf` 文件中的像素数据，并调用绘图函数将像素写入屏幕缓冲区。
2. `libos` 层 应用层的 `fopen`, `fread`, `fseek` 等最终会调用 `libos` 的系统调用包装函数，比如 `_open`, `_read`, `_lseek`。这些函数会通过软中断（`ecall/syscall`）把请求传递给内核（Nanos-lite）。
3. Nanos-lite（内核）Nanos-lite 的 `do_syscall()` 负责处理系统调用请求。对于文件操作（如 `SYS_open`, `SYS_read`, `SYS_lseek`），Nanos-lite 会调用自己的文件系统代码（如 `fs_open`, `fs_read`, `fs_lseek`），在镜像中查找并读取 `mgo.mkf` 文件的数据。对于图像绘制，`PAL_DrawImage` 最终会调用写显存的接口，可能通过系统调用（如 `SYS_write` 到帧缓冲区）。
4. AM（Abstract Machine）Nanos-lite 的底层 I/O 操作（如写显存、读设备）会调用 AM 层的接口，比如 `io_write(AM_GPU_FB_DRAW, ...)`。AM 层负责把这些操作转化为对模拟器（NEMU）的设备访问。
5. NEMU（模拟器）NEMU 仿真 CPU 指令和外设，接收来自 AM 层的设备访问请求。当 AM 层写显存时，NEMU 会更新模拟的帧缓冲区，并在窗口中显示像素变化（即仙鹤动画）。NEMU 也负责模拟文件系统，响应 `mgo.mkf` 的读取请求，把文件内容送到应用层。
6. Trace 工具的作用 使用 `strace` 可以看到应用层每次发起的系统调用（如 `open/read/write`）。使用 `nanos-lite` 的 `trace` 可以看到内核如何分发和处理这些调用。使用 NEMU 的设备 `trace` 可以看到显存、文件系统等外设

的访问情况。